

From Prompt to Exploit: Security Vulnerabilities in LLM-Generated Smart Contracts

CSC 5991 — Final Project Presentation

Anik Tahabilder Dong Vo

Wayne State University

April 21, 2026

Outline

- 1 Background on Smart Contracts
- 2 Motivation
- 3 Research Questions
- 4 Methodology
- 5 Results (RQ1–RQ4)
- 6 Root Cause Analysis
- 7 Human Review
- 8 Mitigation and Proposed Pipeline
- 9 Conclusions

Background: Smart Contracts

- Programs on the blockchain (mostly Ethereum)
- Written in Solidity
- Power DeFi, NFTs, DAOs, bridges
- Hold **hundreds of billions of dollars**

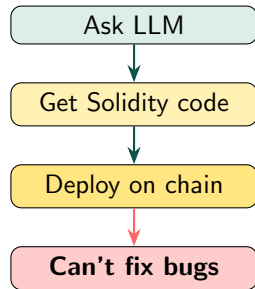
Three key properties:

- **Immutable** — can't be patched
- **Public** — anyone can read the code
- **Adversarial** — anyone can call it

One bug → instant exploit.

Motivation

- Developers use ChatGPT, Copilot, Claude to write Solidity
- \$3.8B stolen from smart contract bugs in 2022
- Once deployed — **no patches**
- Perry et al. (CCS 2023): AI users write less secure code but **feel safer**
- Nobody has measured this at scale



Research Questions

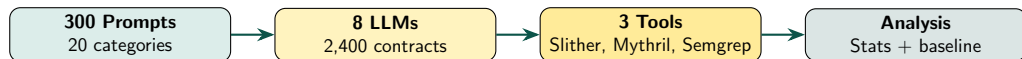
RQ1. How common are bugs? Do LLMs differ?

RQ2. What kinds of bugs? Any LLM-specific patterns?

RQ3. Do tricky prompts make it worse?

RQ4. How does LLM code compare to real Ethereum code?

Methodology and Dataset



General LLMs: GPT-4o, GPT-5, Claude 3.5
Sonnet, Gemini 2.5

Code LLMs: DeepSeek, Qwen 2.5, CodeLlama,
Codestral

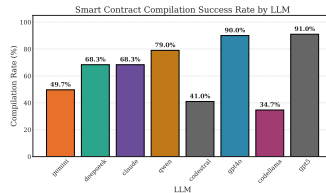
Baseline: 241 real contracts from Etherscan
(Uniswap, Aave, Compound, Lido, ...)

2,493 unique bugs across 1,566 compiled contracts

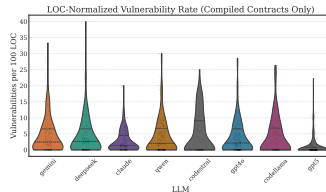
RQ1: Vulnerability Prevalence

Severity	Count	Contracts
High	804	419 (26.8%)
Medium	619	397 (25.4%)
Low	987	584 (37.3%)
Total	2,410	856 (54.7%)

- **Half** of contracts buggy
- **1 in 4** high-severity



Compile rates: 35%–91%



Bug density per 100 LOC

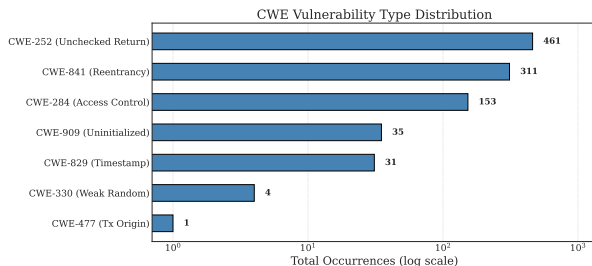
RQ1: Cross-LLM Comparison

Model	Type	Compiles	% Vuln	Mean Bugs	Bugs/100 LOC	Avg LOC
GPT-5	General	91.0%	26.7%	0.63	0.99	49
GPT-4o	General	90.0%	62.2%	1.65	3.86	52
Qwen 2.5 Coder	Code	79.0%	58.2%	1.40	4.10	42
Claude 3.5 Sonnet	General	68.3%	69.3%	3.22	2.75	113
DeepSeek Coder V2	Code	68.3%	62.0%	1.44	4.24	38
Gemini Flash-Lite	General	49.7%	66.4%	1.85	4.25	55
Codestral 22B	Code	41.0%	48.0%	1.06	4.88	24
CodeLlama-34B	Code	34.7%	48.1%	0.95	4.09	26

- Big differences: Kruskal-Wallis $H = 132.7$, $p < 10^{-25}$
- **GPT-5 wins** — best compile, lowest density
- General vs. code-specialized — no clear winner per LOC

RQ2: Vulnerability Taxonomy

#	Type	Count
1	Unchecked Return	461
2	Reentrancy	346
3	Access Control	173
4	Coding Standards	147
5	Incorrect Calc.	85
6	Timestamp Dep.	52
7	Uninitialized State	35



- Top 2: unchecked returns, reentrancy
- All 8 LLMs share the same top CWEs

RQ2: LLM-Specific Patterns

1. Phantom Guards

Empty `nonReentrant` — looks safe, isn't.

2. Unnecessary Reimplementations

SafeMath on Solidity 0.8+ (4% of contracts).
DeepSeek worst.

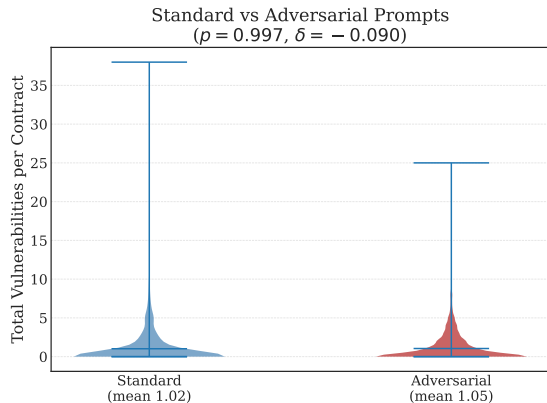
3. Context Window Degradation

In long contracts, **4.1×** **more** risky code in the second half.

Listing 1: Phantom guard

```
1 // LLM-generated (VULNERABLE)
2 modifier nonReentrant() {
3     _; // No lock!
4 }
5
6 function withdraw(uint amt)
7     nonReentrant {
8     payable(msg.sender)
9         .call{value: amt}("");
10    balances[msg.sender] -= amt;
11 }
```

RQ3: Adversarial Prompt Robustness



Short answer: No.

- $p = 0.965$ (not significant)
- Cliff's $\delta = -0.056$ (negligible)
- Adversarial $\approx$ standard

“Optimize gas,” “keep it minimal,” “no dependencies” — none hurt security.

Key insight

Bugs come from **training data**, not prompts. You can't prompt your way to safe code.

RQ4: LLM vs. Human-Written Contracts

Metric	Human	LLM
Contracts	241	2,400
Compiled	54.8%	65.2%
% with bugs	82.6%	54.7%
Mean LOC	446.7	52.3
Bugs/100 LOC	2.69	0.99–4.88
High-sev %	16.0%	15.7%

Human: Uniswap, Aave, Compound, Gnosis, Lido

- Bug density **similar** per LOC
- High-severity rates **almost identical**
- Human contracts **8× larger**

But:

- Human code was audited, tested, bountied, monitored
- LLM code usually just ships

The danger is **skipping the pipeline.**

Root Cause Analysis

1. Training data bias

- All 8 LLMs share top CWEs
- Trained on pre-2023 Solidity
- Adversarial null → **intrinsic**

2. Compilation failures

- Bad `@openzeppelin` imports
- Markdown in code
- CodeLlama: 65% syntax fails

3. Model fingerprints

- Claude: equality checks
- CodeLlama: bad ERC-20
- All: missing zero checks

4. Category matters

- Worst: Proxy, Yield, DEX
- Safest: Access Control, ERC-20

Human Review Validation

We manually reviewed 195 of the 241 baseline contracts.

Metric	Result
Reviewed	195
With issues	161 (82.6%)
Total issues	310
High severity	117 (37.7%)
Medium severity	163 (52.6%)

Top issues:

- 1 Access Control (72)
- 2 Unchecked Returns (67)
- 3 Integer Overflow (16)
- 4 Reentrancy (16)
- 5 External Calls (14)

Humans catch what Slither misses:
signature replay, centralization, storage collision.

Tools + humans > tools alone.

Mitigation: Can a Security Prompt Fix It?

What we tried:

Add a “use checks-effects-interactions, validate inputs, ReentrancyGuard, SafeERC20, ...” preamble to 20 prompts (Claude).

Hypothesis: explicit security cues help.

Result: they don't.

Metric	Plain	Prefix
Mean bugs	2.20	8.73
Got worse	13 / 20	
Got better	5 / 20	

Why: prefix adds more code → more findings (mostly low-severity).
High-severity bugs actually went **down by 2**.

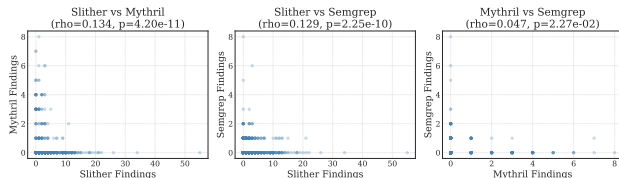
Takeaway

Prompts aren't the fix. The **pipeline** is.

No Single Tool Catches Every Bug

Prompts can't fix the code (slide 15). So we need stronger detection — but can one tool do it?

**Agreement between the 3 analyzers
(Spearman ρ):**

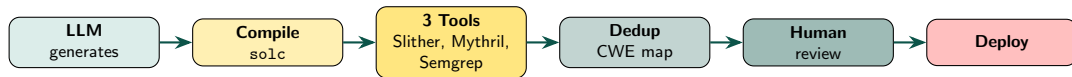


Tool pair	ρ
Slither vs. Mythril	0.134
Slither vs. Semgrep	0.129
Mythril vs. Semgrep	0.047

- All $\rho < 0.15$ — near-zero overlap
- Each tool finds bugs the others miss
- **Single-tool audits are insufficient**

Conclusion: one tool isn't enough — the audit pipeline needs all three.

Proposed: LLM-Aware Audit Pipeline



Why each step:

- 35% doesn't compile → gate first
- Tools disagree ($\rho \approx 0.13$) → use 3
- Logic bugs hide from tools → humans

End-to-end precision/recall on a ground-truth corpus remains future work.

What we measured (1,566 contracts):

- 3 tools: **100%** coverage vs 93.9% Slither-only
- Dedup: **23%** reduction (3,133 → 2,410)
- Human step validated on baseline only

Implications and Contributions

For developers:

- ① Never deploy without auditing
- ② Don't trust security prompts
- ③ Prefer GPT-5
- ④ Extra care: DeFi, long code
- ⑤ Watch the second half

Our contributions:

- ① Largest study: 2,400 contracts, 8 LLMs
- ② Adversarial null result (new)
- ③ 3 novel LLM patterns
- ④ Root cause analysis
- ⑤ Real Ethereum baseline + review

Limitations and Future Work

Limitations:

- Snapshot — models change
- Static tools have false positives
- 8 LLMs, not all
- Solidity only
- Single-shot, no multi-turn

Future work:

- Rust/Solana, Move/Aptos
- Fuzzing, runtime analysis
- Multi-turn developer sessions
- LLM-aware static tools
- Security fine-tuning

Summary

2,400	contracts from 8 LLMs
2,493	unique bugs
54.7%	of compiled have real bugs
26.8%	have high-severity
$p = 0.965$	adversarial doesn't matter
0.99–4.88	LLM bugs per 100 LOC
2.69	human baseline density

LLM code isn't worse than human code —
but it skips the audit pipeline.

Thank you! Questions?

github.com/atahabilder1/AiContractCheck